

The QED Unified Symmetry Stack

Mathematics and implementation, down to the lowest level

(post–Stage-11 reference, July 2026)

QED exact-diagonalization library
repo `ze-bang/QED`, branch `main`

July 2026

Abstract

This is the complete technical reference for the QED exact-diagonalization engine after the Stage 9–11 consolidation. The pipeline is one sentence — *given a Hamiltonian: find the symmetries, build the basis, do the matvec, run the Lanczos* — and after the consolidation every step has exactly one implementation. This document derives the mathematics each step rests on (monomial group actions, representative bases and their norms, the $A' = A \rtimes \mathbb{Z}_2$ flip extension, little co-groups and isotypic projection, time-reversal folds, Krylov convergence and the ghost-Ritz mechanism, finite-temperature estimators, continued-fraction spectral functions) and pairs each derivation with the code that implements it, quoted from the tree. The final sections cover the cross-cutting guards (covering sum rules, commutator probes, residual gates, content-hash verification), the two July hardenings, and the measured validation numbers.

Contents

1	The problem and the design invariant	2
1.1	Setting	2
1.2	The invariant that generates every layer	2
1.3	The seven layers	3
2	Group action and the representative basis	3
2.1	Monomial action on bitstrings	3
2.2	Orbits, stabilizers, projected states	3
2.3	Matrix elements: the representative walk	4
2.4	One matvec representation, two sub-modes	5
3	The flip extension $A' = A \rtimes \mathbb{Z}_2$	6
4	The little-group engine (lane A)	7
4.1	Stars: residue conjugation and the union–find fold	7
4.2	Monomial little co-group	7
4.3	Abstract decomposition and isotypic bases	8
4.4	Time reversal: the fold that never projects	8
4.5	Routing (lane A vs lane B)	8
5	Lane B: the abelian composition lane	9

6	Krylov numerics: Lanczos, ghosts, and the July 2026 fix	9
6.1	The kernel	9
6.2	The ghost mechanism	10
6.3	The bug and the fix (commit 7651683)	10
7	Finite temperature and spectra	11
7.1	Thermal kernels	11
7.2	Ground-state dynamical structure factors	11
8	Backends, dispatch, and the GPU dimension floor	12
9	One operator template, one parameter converter	13
9.1	The operator hierarchy	13
9.2	The one converter	13
10	Cross-cutting guards and content-addressed caching	14
10.1	Correctness never rides on bookkeeping	14
10.2	Hash quality is a correctness concern	14
11	The Stage-11 consolidation ledger	15
12	Validation numbers	15

1 The problem and the design invariant

1.1 Setting

Spin- $\frac{1}{2}$ models on N -site clusters. The computational basis is the σ^z product basis, identified with bitstrings $s \in \{0, 1\}^N \cong \{0, \dots, 2^N - 1\}$; site i is up iff bit i of s is set. The Hamiltonian is a sum of one-, two- and three-body terms,

$$H = \sum_t c_t O_t, \quad O_t \in \{S_i^z, S_i^\pm, S_i^z S_j^z, S_i^+ S_j^-, S_i^\pm S_j^\pm, \dots\},$$

stored as structure-of-arrays term lists (`ed/matvec/term.storage.h`). Every solver consumes H only through its action $v \mapsto Hv$ (“matvec”); no full matrix is ever required, though *reduced* sector matrices are assembled when they fit a budget (§2.4).

At the frontier ($N = 36$, half filling) the $U(1)$ block has dimension $\binom{36}{18} \approx 9.1 \times 10^9$: a single complex vector is ~ 145 GB. Everything below exists to divide that number by the symmetry group and to keep the working set at $O(\#\text{representatives})$.

1.2 The invariant that generates every layer

Proposition 1.1 (project vs. fold). *Let $\mathcal{H} = \bigoplus_b \mathcal{H}_b$ be a decomposition preserved by H , and let U be a unitary with $[H, U] = 0$. (i) If $U\mathcal{H}_b = \mathcal{H}_b$, then $U|_{\mathcal{H}_b}$ generates a further orthogonal decomposition of $\mathcal{H}_b$ into eigenspaces of U (projection: smaller blocks). (ii) If $U\mathcal{H}_b = \mathcal{H}_{b'}$ with $b' \neq b$, then $H|_{\mathcal{H}_{b'}} = UH|_{\mathcal{H}_b}U^\dagger$ is unitarily equivalent to $H|_{\mathcal{H}_b}$ (fold: one solve, copied spectrum, multiplicity 2). (iii) An antiunitary K with $KHK^{-1} = H$ never projects: K has no eigenspaces in the linear sense; its entire exploitable content is the fold $H|_{K\mathcal{H}_b} \cong \overline{H|_{\mathcal{H}_b}}$, which for Hermitian H is isospectral.*

The stack is the maximal assignment of these three roles:

$$\text{one solve} = \underbrace{(U(1) \, n_{\uparrow} \text{ or } S^z\text{-parity})}_{\text{subspace}} \times \underbrace{(A' = A \rtimes \mathbb{Z}_2^{\text{flip}}) \otimes P_{k_0}}_{\text{projector}} \times \underbrace{(\text{stars} \cdot \text{TR} \cdot \text{transport})}_{\text{folds}}.$$

1.3 The seven layers

	Layer	The one implementation
L7	user verbs	<code>workflow.py</code> / <code>thermal.py</code> / <code>spectral.py</code> ; parameters via <code>_params.py</code> $\Rightarrow$ <code>ed.config_adapter</code>
L6	symmetry discovery	<code>discovery.py</code> (nauty automorphisms $\rightarrow$ abelian clique + residues)
L5	routing	<code>point_group_routing.py</code> (<i>the</i> project-vs-fold decision)
L4	two lanes	A: <code>little_group_solve.cpp</code> ; B: <code>sector_plan.h</code> + workflow bindings
L3	basis artifacts	<code>compiled_group.h</code> $\rightarrow$ <code>orbit_table.h</code> $\rightarrow$ <code>rep_sector_data.h</code> (lazy-only builders, <code>sector_set.h</code>)
L2	matvec	the CSR-free representative kernel, host + device; reduced-CSR sub-mode, budget-gated
L1	kernels	<code>ed/krylov/lanczos_kernel.h</code> ; thermal kernel headers; <code>orchestrator.cpp</code> + <code>select_backend</code>

2 Group action and the representative basis

2.1 Monomial action on bitstrings

The exploited symmetries act on the computational basis by *site permutations composed with global bit-flips*. An element $g = (\pi_g, f_g)$, with $\pi_g \in S_N$ and $f_g \in \{0, \text{all-ones}\}$, acts as

$$U_g|s\rangle = |g(s)\rangle, \quad g(s) = \pi_g(s) \oplus f_g,$$

a pure permutation of basis states (no phases: $\prod_i \sigma_i^x$ maps a product state to a product state with coefficient +1). The group G is therefore realized by 0/1 monomial matrices, and everything downstream — orbits, stabilizers, norms — is combinatorics on bitstrings.

Implementation. `ed/symmetry/compiled_group.h` compiles each element to a byte-lookup table: the permutation is applied one input byte at a time (bpw = $\lceil N/8 \rceil$ table loads), then one XOR applies the flip:

```

// Action of element 'g' on state 's'. BPW table loads + one XOR.
[[nodiscard]] inline std::uint64_t
apply(std::uint64_t s, std::size_t g) const noexcept {
    const std::uint64_t* lut_g = lut_.data() + g * stride_;
    std::uint64_t out = 0;
    for (int b = 0; b < bpw_; ++b) {
        out |= lut_g[(static_cast<std::size_t>(b) << 8)
                     + ((s >> (8 * b)) & 0xFFu)];
    }
    return out ^ flips_[g];
}

```

The bit-identity contract (`apply(s,g) == applyPermutation(s, perm[g]) ^ flips[g]`) is pinned against the scalar reference by `test_compiled_group.cpp` ($\sim 17k$ assertions).

2.2 Orbits, stabilizers, projected states

Fix an abelian group A (the maximal abelian subgroup of the automorphism group, extended by the flip where admissible, §3) with characters $\chi_k : A \rightarrow U(1)$. For each orbit $\mathcal{O}_r = \{a(r) : a \in A\}$

choose the representative $r = \min \mathcal{O}_r$ (integer order on bitstrings). The projected state in irrep k is

$$|r; k\rangle = \frac{1}{\sqrt{|A| \nu_r(k)}} \sum_{a \in A} \overline{\chi_k(a)} U_a |r\rangle, \quad \nu_r(k) = \sum_{h \in \text{Stab}(r)} \chi_k(h), \quad (1)$$

where $\text{Stab}(r) = \{h \in A : h(r) = r\}$. Since A is abelian and χ_k is a homomorphism, $\nu_r(k) = |\text{Stab}(r)|$ if χ_k restricts trivially to $\text{Stab}(r)$ and 0 otherwise — the *survival condition*. The basis of the $(n_\uparrow, k)$ sector is $\{|r; k\rangle : \nu_r(k) > 0\}$, of dimension $D_k = \#\{\text{surviving reps}\}$, and Burnside’s lemma gives the covering sum rule used as a runtime tripwire (§10):

$$\sum_k D_k = \binom{N}{n_\uparrow}. \quad (2)$$

Implementation. A single fused scan (`ed/symmetry/orbit_table.h`) enumerates representatives *and* stabilizers per (subspace, G) pair — the `OrbitTable`. The scan walks the subspace in ascending state order; for each state s it computes the $|G|$ images with the byte-LUT `apply`, early-exits the moment any image is $< s$ (then s is not a representative — this is the dominant, cheap branch), and otherwise records s as a representative together with the stabilizer bitmask $\{g : g(s) = s\}$ collected in the same pass. A closed-form popcount prefilter skips states whose bit count cannot match the subspace before any group work. Stabilizer masks are deduplicated (`StabDedup`): lattices have few distinct stabilizer subgroups, so per-rep storage is an index into a small table. The whole artifact is cached in-process by content hash and on disk as `<lattice>/basis_cache/sym_v2/<hash>.otab` (§10.2); a warm $N=28$ run loads 1.43M representatives from a 14 MB file instead of rescanning.

Per irrep, the $O(\#\text{reps})$ view is a `RepSectorData`: sorted `reps[]`, `inv_norms[i] = \nu_{r_i}(k)^{-1/2}` evaluated in closed form from the stored stabilizer ($\nu = \sum_{h \in \text{Stab}} \chi_k(h)$, zero-norm reps dropped — the survival condition), the $|A'|$ characters, the flattened permutations, and (when the flip is engaged) per-element XOR masks. Construction is *lazy-only* since Stage 11c-1: the builders in `sector_set.h` hand every sector a `RepSectorData` provider (`SectorBasis::configureRepLazy`), and the legacy per-sector “orbit CSR” exists only as an on-demand product for orbit-*data* consumers (`SectorBasis::build` behind `ensureHostCsr`: the dense-assembly decline path and cross-sector observables).

2.3 Matrix elements: the representative walk

Let a Hamiltonian term act as $O_t |r_i\rangle = c_t |s'\rangle$ (off-diagonal terms connect one basis state to one basis state). Projecting $H |r_i; k\rangle$ back onto the sector basis:

Proposition 2.1 (rep-walk matrix element). *With $r_b = \min_a a(s')$ the representative of the destination orbit and j its index,*

$$\langle r_j; k | O_t | r_i; k \rangle = c_t \cdot \frac{1}{\sqrt{\nu_i}} \cdot \overline{\beta_{s'}} \cdot \frac{1}{\sqrt{\nu_j}}, \quad \beta_{s'} = \sum_{\{h \in A : h(s') = r_b\}} \chi_k(h). \quad (3)$$

The sum over the $|A|$ -fold orbit walk of the naive projector, and its global $1/|A|$ weight, collapse into this single representative term.

Sketch. Expand both projected states via (1); the double group sum $\sum_{a, a'}$ reduces by substitution $h = a'^{-1}a$ to $|A|$ times a sum over the coset $\{h : h(s') = r_b\}$; the $|A|$ cancels against the two $1/\sqrt{|A|}$ normalizations. $\square$

Implementation. The kernel-facing form is `index_and_projection`: given a connected state, return the destination index j and the scalar $\overline{\beta_s} \nu_j^{-1/2}$ (the row factor $\nu_i^{-1/2}$ is supplied as the kernel's `pre_phase`). The host implementation (`ed/matvec/rep_symmetry_basis_policy.h`) computes all $|G|$ images once into a stack buffer; one pass finds the minimum, one accumulates the character sum:

```
[[nodiscard]] inline std::int64_t
index_and_projection(std::uint64_t state, Complex& proj_out) const noexcept {
    if (n_up >= 0 && __builtin_popcountll(state) != n_up) return -1;
    if (group_size <= kMaxG) { // kMaxG = 256
        std::uint64_t images[kMaxG];
        std::uint64_t rb = state;
        for (int g = 0; g < group_size; ++g) {
            images[g] = apply_perm(state, g);
            if (images[g] < rb) rb = images[g];
        }
        const std::int64_t k = index_of_rep(rb);
        if (k < 0) return -1;
        double acc_re = 0.0, acc_im = 0.0;
        for (int h = 0; h < group_size; ++h) {
            if (images[h] == rb) {
                acc_re += characters[h].real(); // conj: +real
                acc_im -= characters[h].imag(); //      -imag
            }
        }
        const double s = inv_norms[static_cast<std::size_t>(k)];
        proj_out = Complex(acc_re * s, acc_im * s);
        return k;
    }
    /* |G| > 256: two-pass, buffer-free fallback (same math) */
    ...
}
```

The reverse lookup `index_of_rep` is a binary search on the sorted `reps` (zero memory), upgraded to $O(1)$ by two budget-gated structures: a per-sector dense table indexed by the *combinadic rank*

$$\text{rank}(s) = \sum_{m=1}^{n_{\uparrow}} \binom{b_m}{m}, \quad b_1 < b_2 < \dots \text{ the set-bit positions of } s, \quad (4)$$

or — the production default at scale — *one* shared rank $\rightarrow$ shared-rep-index table per $(N, n_{\uparrow})$ subspace co-owned by every irrep sector (`SharedRankLookup`; 77 GB $\rightarrow$ 4.8 GB aggregate at $N = 32$). The identical arithmetic runs on the GPU (`DeviceRepSymmetryBasisPolicy`, `device.basis_policy.cuh`); full-space sectors ($n_{\uparrow} = -1$) use the identity rank.

2.4 One matvec representation, two sub-modes

Since Stage 11c-2b the representative kernel is the *only* symmetry matvec on both backends. Its two sub-modes trade memory for speed:

- **RepReducedCsr (default).** Assemble the reduced sector matrix once by running (3) over every representative row (`ed/matvec/reduced_symmetry_csr.h`; OpenMP-parallel over rows), then every iteration is a plain $O(\text{nnz})$ SpMV. The build costs one full group walk $\approx |G|$ SpMV-equivalents, so the breakeven against the walk is exactly iterations = $|G|$ — always won by an iterative solver. Budget-gated per sector by an up-front upper bound against

ED_SYM_SECTOR_CSR_BUDGET_GIB (default 8), on *both* lanes: an over-budget frontier sector falls back by itself.

- **RepStream** (ED_SYM_REDUCED_CSR=0, and the automatic over-budget fallback): regenerate (3) arithmetically on every apply. $O(\#reps)$ memory — the $N = 36$ regime — at $\sim |G| \times$ per-apply cost.

The budget gate is one consolidated helper, shared by both lanes (ed/planner/sym_matvec_policy_hook.h):

```
[[nodiscard]] inline bool sector_csr_within_budget(
    std::uint64_t dim, std::uint64_t terms_per_row) noexcept {
    const std::uint64_t est_bytes =
        dim * terms_per_row * (16u /* complex value */ + 4u /* col idx */)
        + (dim + 1) * 8u /* row ptr */;
    double budget_gib = 8.0;
    if (const char* v = std::getenv("ED_SYM_SECTOR_CSR_BUDGET_GIB")) {
        const double b = std::atof(v);
        if (b > 0.0) budget_gib = b;
    }
    return static_cast<double>(est_bytes)
        <= budget_gib * static_cast<double>(1ULL << 30);
}
```

The builder `build_reduced_symmetry_csr_rep` (ed/matvec/reduced_symmetry_csr.h) assembles row r by applying every Hamiltonian term to the single representative rep_r and resolving each connected state through `index_and_projection` — *verbatim* the same element math as the streaming kernel, which is what makes the two sub-modes bitwise comparable (pinned by `test_reduced_symmetry_csr.cpp` and the integration strategy-parity test). Rows are independent, so the build is OpenMP-parallel. Dense consumers (`solver="full"` on a sector) densify the same reduced CSR; the eager orbit-walk dense assembler was deleted with the orbit lane. On the GPU the resident mirror holds `reps + inv_norms + perms +  $\chi$` (~ 600 MB at $N = 32$) and regenerates the group action inside the kernel; measured end-to-end at $N = 24$: rep mirror 4.8s vs the retired orbit-CSR mirror's 18.2s.

3 The flip extension $A' = A \rtimes \mathbb{Z}_2$

The global spin flip $X = \prod_i \sigma_i^x$ commutes with every site permutation, so when $[H, X] = 0$ it extends the *abelian* factor — never the point-group residue: $A' = A \rtimes \langle X \rangle$, realized on bitstrings by the all-ones XOR mask.

Lemma 3.1 (closure rule). *X preserves the diagonal subspace iff the subspace is flip-invariant: the $n_\uparrow$ block only at $n_\uparrow = N/2$; an S^z -parity half only for even N (X changes $n_\uparrow$ by $N - 2n_\uparrow$, of parity $N \bmod 2$); the full space always. Elsewhere X is a block mapper and contributes only transport: $\text{spec } H|_{n_\uparrow} = \text{spec } H|_{N-n_\uparrow}$ (Prop. 1.1 (ii)).*

This rule is stated *once* (`flip_subspace_admissible`, ed/symmetry/spin_flip.h) and consumed by the engine and every binding gate. Where admissible, the extended characters are *synthesized*, never re-decomposed: elements are laid out $[A, A \cdot X]$ and

$$\chi'_{k,s}(aX^f) = \chi_k(a)(\pm 1)^f, \quad s \in \{+, -\}, \quad (5)$$

doubling the irrep count ($k_{\text{ext}} = k + s n_{\text{irr}}$) and halving the big sector:

```
// Extended-irrep characters chi'_{k,s}(a + f|A|) = chi_k(a) * (s ? -1 : +1)^f.
[[nodiscard]] std::vector<Complex>
```

```

characters_for(const EngineContext& cx, int k_ext) {
    const auto& base =
        cx.giA.irreps[static_cast<std::size_t>(k_ext % cx.n_irr_raw)].character;
    std::vector<Complex> chi(base.begin(), base.end());
    if (cx.flip_half) {
        const double s = (k_ext / cx.n_irr_raw == 0) ? 1.0 : -1.0;
        chi.reserve(2 * base.size());
        for (const Complex& c : base) chi.push_back(s * c);
    }
    return chi;
}

```

Everything downstream — norms (1), the walk (3), the reduced CSR, the GPU mirror — is automatically flip-correct because the `CompiledGroup` carries the XOR masks inside `apply` and the characters carry the sign. Note that the covering sum rule (2) is *unchanged* by the flip: the flip halves sectors, not the subspace — this makes it the Stage-9a tripwire rather than a quantity to “fix”.

4 The little-group engine (lane A)

The full automorphism group is $G = A' \rtimes P$ with P the retained non-abelian residue (e.g. reflection-s/rotations). Lane A exploits P *maximally*: block-mapping residues fold sectors into stars; residues that fix a sector are projected via its little co-group. All of this lives in `src/solvers/cpu/little_group_solve.cpp` (~ 1.4 kLOC).

4.1 Stars: residue conjugation and the union–find fold

A residue p normalizes A' , so conjugation $c_p(a) = p^{-1}ap$ permutes the abelian irreps: $\chi_k \mapsto \chi_k \circ c_p$. Sectors k and k^p are unitarily equivalent (Prop. 1.1 (ii) with $U = U_p$), with equal dimensions by conjugation-invariance of $|\nu_r|$. One union–find over the extended irreps, with edges from every residue map *and* from time reversal (§4.4), partitions the sector space into **stars**; each star is solved once at its representative k_0 with multiplicity $|\text{star}|$. Under the flip, residues act parity-diagonally ($\chi'_{k,s} \mapsto \chi'_{k^p,s}$): stars never mix flip parities.

4.2 Monomial little co-group

The little co-group of k_0 is $L = \{p \in P : \chi_{k_0} \circ c_p = \chi_{k_0}\}$ modulo A' . Its elements act on the representative basis by *monomial matrices*:

Proposition 4.1. *For $p \in L$ and a surviving representative r_i , let $s = \pi_p(r_i)$, let $r_b = \min_a a(s)$ with witness a^* ($a^*(s) = r_b$), and let j index r_b . Then*

$$U_p |\psi_i(k_0)\rangle = \overline{\chi_{k_0}(a^*)} \frac{\nu_j^{1/2}}{\nu_i^{1/2}} |\psi_j(k_0)\rangle,$$

and the coefficient has unit modulus. Coset representatives suffice: $M_{ap} = \chi_{k_0}(a) M_p$.

```

// Monomial action of residue p on the k0 rep basis (build_monomial).
for (std::size_t i = 0; i < dim; ++i) {
    const std::uint64_t s = applyPermutation(rd.reps[i], p);
    std::uint64_t rb      = ~std::uint64_t{0};
    std::size_t   astar = 0;

```

```

    for (std::size_t a = 0; a < nA; ++a) { // byte-LUT walk over A'
        const std::uint64_t img = cx.cg.apply(s, a);
        if (img < rb) { rb = img; astar = a; }
    }
    const auto it = std::lower_bound(rd.reps.begin(), rd.reps.end(), rb);
    if (it == rd.reps.end() || *it != rb) return false; // fallback
    const std::size_t j = static_cast<std::size_t>(it - rd.reps.begin());
    // U_p |psi_i> = chi(b) (N_j/N_i) |psi_j>, b = a*{-1}
    const Complex ph = std::conj(chi[astar])
        * (rd.inv_norms[i] / rd.inv_norms[j]);
    if (std::abs(std::abs(ph) - 1.0) > 1e-8) return false; // must be unit
    out.to[i] = static_cast<std::int32_t>(j);
    out.phase[i] = ph / std::abs(ph);
}

```

Each accepted monomial is then *numerically validated* by a random-vector commutator probe $\|[M_p, H_{k_0}]x\|/\|H M x\| < 10^{-8}$ (`monomial_commutes`): a convention error or a residue that does not truly fix the sector costs only the refinement, never correctness.

4.3 Abstract decomposition and isotypic bases

The monomials $\{M_l\}_{l \in L}$ realize L only projectively in general; the engine therefore decomposes the *abstract* group from its multiplication table (little co-groups need no faithful permutation realization). `decompose_irreps_tables` computes the irreducible characters χ_σ of L from the regular representation's commutant; the isotypic projectors

$$P_\sigma = \frac{d_\sigma}{|L|} \sum_{l \in L} \overline{\chi_\sigma(l)} M_l \quad (6)$$

are applied per *index orbit* of the monomial permutation structure, and sparse orthonormal bases W_σ of their ranges are extracted by small SVDs. The block solved is $W_\sigma^\dagger H_{k_0} W_\sigma$ with eigenvalue multiplicity $|\text{star}| \times d_\sigma$; the covering check $\sum_\sigma (\# \text{cols } W_\sigma) d_\sigma = \dim k_0$ guards completeness. **Fallback ladder**: projective factor system, non-closing residues, failed probe, incomplete covering $\Rightarrow$ solve the plain k_0 block — less reduction, never wrong physics.

4.4 Time reversal: the fold that never projects

For H real in the computational basis, complex conjugation K satisfies $K H K = H$. In the representative basis, K maps sector k to $\bar{k}$ (characters conjugate), giving $H_{\bar{k}} = \overline{H_k}$ — isospectral, so $k \leftrightarrow \bar{k}$ edges join the star union-find. Inside a self-conjugate star whose χ_{k_0} is *all-real* (hence H_{k_0} real and the monomial phases real), conjugation maps the isotypic component σ to σ^* : $\overline{W_\sigma}$ spans the σ^* isotypic, so conjugate pairs solve *once* at doubled multiplicity. Both folds are exact copies, guarded (equal W column counts, covering accounting) and revert to solving both on any doubt.

4.5 Routing (lane A vs lane B)

One decision point, `resolve_projection_lane` (`point_group_routing.py`): `point_group="auto"` (default) projects every eigenvalue-only call; declines — eigenvectors, `sector=`, sampling methods, an explicit GPU request, no residue, `ED_SYM.LITTLE_GROUP=0` — degrade silently to lane B (the abelian composition lane: same role assignment, fold-heavy, serving vectors/sampling/GPU/per-sector

metadata). "full" raises with the decline reason; no fallback engine exists (the monolithic symmetry-adapted-basis engine survives only as a parity-test oracle). Explicit non-abelian generator lists are closed (cap 4096) and carved into (maximal-abelian core, coset residues) by `split_nonabelian`.

5 Lane B: the abelian composition lane

Lane B serves every consumer the projection engine declines: eigenvector requests, per-sector metadata, sampling methods (FTLM/TPQ), and the GPU. It assigns the *same* roles as lane A but exploits the block-mappers as folds over the abelian sector set rather than projecting little co-groups.

Composition plan (`ed/symmetry/sector_plan.h`). Pure planning functions shared by the thermal flat pool and the ground-state lane: `resolve_symmetry_composition` normalizes the per-call toggles (`spin_flip=`, `time_reversal=` in `{auto, on, off, require}`, one `SymToggle` convention shared with the engine) against term-level detection (`[H, X] = 0` checker; `H`-real checker); `plan_build_window` shrinks the $n_{\uparrow}$ sweep to $n_{\uparrow} \leq N/2$ under flip transport (Lemma 3.1(ii), spectra mirrored to $N - n_{\uparrow}$ with tags restored); `plan_tr_actions` plus the point-group *star maps* (how each residue conjugates the abelian irreps, pure permutation algebra computed in Python) feed *one* union-find (`sector_orbit_canonical`) that folds TR pairing $k \leftrightarrow \bar{k}$ and all residue maps $k \mapsto k^p$ into isospectral orbits — solve one representative per orbit, copy the spectrum. The fold arithmetic is flip-slot aware: synthetic indices $k + s n_{\text{irr}}$ pair within parity blocks.

In-sector flip projection. At $n_{\uparrow} = N/2$ (or on parity halves / the full space) lane B splits each momentum sector into $(k, \pm)$ slots using the same extended characters (5); the flip-projected sectors have *no orbit-CSR form at all* (the provider throws by design) — they exist only in the representative representation, which is why the one-matvec consolidation (§2.4) was a precondition for this machinery rather than an optimization.

Labels. Both lanes emit an honest per-eigenvalue attribution: the engine pools $(k_{\text{raw}}, \text{flip parity}, \sigma, d_{\sigma})$ labels plus the raw-irrep character table (the engine's internal irrep order is *not* the directory sector order; momentum is decoded from the phase of $\chi_k(T)$, convention $e^{-2\pi i k/N}$), and lane B carries per-sector eigenvalue arrays and `SectorTags`. The pooled-vs-tagged fork is a documented output contract, not an accident.

Ground-state arbitration. The cross-irrep bindings locate the ground sector by a two-phase scan (`find_gs_sector_two_phase`): a cheap 40-iteration Lanczos per sector prunes to candidates within a gap window of the running minimum; survivors re-solve at 10^{-12} ; the winner's eigenpair must then pass the residual gate of §7.2 before any weight is computed. Phase-1 failures mark a sector $-\infty$ (never silently dropped), and a non-finite phase-1 sweep falls back to scanning everything.

6 Krylov numerics: Lanczos, ghosts, and the July 2026 fix

6.1 The kernel

One templated implementation, `ed::krylov::lanczos_kernel<Backend>` (`ed/krylov/lanczos_kernel.h`), is *the* Lanczos step for every consumer (ground state, FTLM/LTLM inner loops, continued fractions, thermal pure states). The three-term recurrence

$$\beta_{j+1} v_{j+1} = H v_j - \alpha_j v_j - \beta_j v_{j-1}, \quad \alpha_j = v_j^{\dagger} H v_j, \quad (7)$$

builds $T_m = \text{tridiag}(\beta; \alpha; \beta)$ whose eigenvalues (Ritz values) approximate $\text{spec}(H)$ from the extremes inward. Reorthogonalization policies: `None`, `LocalDGKS $k$` (a ring of the last k basis vectors, re-orthogonalized twice, DGKS-style), `PeriodicCGS2`, `FullCGS2`; options carry `convergence_check` callbacks, `aux_ortho_ptrs` (project out a fixed external set — used by thick-restart Krylov–Schur locking), `dim_cap`, and `keep_basis=false` for value-only runs.

6.2 The ghost mechanism

In exact arithmetic the v_j are orthonormal and every Ritz value of T_m is a Rayleigh–Ritz approximation from a genuine subspace. In finite precision, orthogonality is lost *precisely when a Ritz pair converges* (Paige): the recurrence re-introduces components along converged directions, and T_m then carries *ghost copies* — spurious duplicates of converged eigenvalues, and transiently, spurious values between genuine levels. Two structural consequences matter for a *block* solver that requests the lowest k :

1. ghosts can sit *exactly on* true levels, so a value-membership test passes while the returned *multiset* is wrong (duplicated levels displace true ones from the k -window);
2. near subspace exhaustion ($m \rightarrow \text{dim}$, unavoidable on small blocks) ghost formation is generic even with ring reorthogonalization.

6.3 The bug and the fix (commit 7651683)

The little-group per-block lowest- k solver, `solve_block_lowest`, had delegated to a legacy wrapper with an iteration budget `max_it = 2k + 40` and *no convergence guard* — on near-degenerate blocks every block ran to its cap and returned whatever Ritz values T_{60} held. Verified both ways on the 4×4 J_1 – J_2 model ($J_2 = 0.15$, $n_{\uparrow} = 8$, $k = 10$): the pre-fix body returns the multiset $\{E_0^{\times 2}, E_1^{\times 2}, E_2, (-8.2328)^{\times 5}\}$ against the true $\{E_0, E_1, E_2, (-8.2328)^{\times 4}, (-8.0992)^{\times 3}\}$ — multiset error 1.4; the fixed body reproduces the dense window to 1.3×10^{-13} .

The replacement: below the crossover `max(dense_max_dim, 64k)` a dense values-only eigensolve (near-exhaustion territory: exact and effectively free); above it, the kernel with `LocalDGKS3` (ring 8), no stored basis, a real budget `max(40k, 400)`, and an *all- k -lowest Ritz stationarity gate* — the shared smallest-only predicate would strand the upper levels of the window:

```
kopts.convergence_check =
    [prev, kk] (const std::vector<double>& alpha,
               const std::vector<double>& beta) -> bool {
        const int m = static_cast<int>(alpha.size());
        if (static_cast<std::size_t>(m) < kk + 2) return false;
        /* assemble T_m, dense-solve values-only (Eigen) ... */
        std::vector<double> cur(es.eigenvalues().data(),
                               es.eigenvalues().data() + nk);
        bool conv = prev->size() == cur.size();
        for (std::size_t i = 0; conv && i < cur.size(); ++i) {
            const double den = std::max(std::abs(cur[i]), 1e-300);
            conv = std::abs(cur[i] - (*prev)[i]) / den < 1e-11;
        }
        *prev = std::move(cur);
        return conv;
    };
    // every requested level stationary
kopts.convergence_check_interval = 10;
```

The regression pin (`test_little_group_lowest_window.py`) compares the pooled lowest- k against the pinned dense reference *as sorted multisets* — element-wise on sorted arrays, so one ghost duplicate shifts every subsequent entry.

7 Finite temperature and spectra

7.1 Thermal kernels

All backend-templated headers under `ed/thermal/`, driven by `ed::workflows::thermal:`

FTLM (finite-temperature Lanczos): for R random states $|r\rangle$ with per-state Krylov eigenpairs $(\varepsilon_j^{(r)}, |\psi_j^{(r)}\rangle)$,

$$Z \approx \frac{D}{R} \sum_{r=1}^R \sum_j e^{-\beta \varepsilon_j^{(r)}} |\langle r | \psi_j^{(r)} \rangle|^2, \quad \langle H \rangle \approx \frac{D}{RZ} \sum_{r,j} \varepsilon_j^{(r)} e^{-\beta \varepsilon_j^{(r)}} |\langle r | \psi_j^{(r)} \rangle|^2. \quad (8)$$

LTLM refines the low- T end from a single ground-state-seeded chain. **mTPQ** (microcanonical thermal pure quantum): iterate $|\phi_{m+1}\rangle \propto (L - \hat{h})|\phi_m\rangle$ with $\hat{h} = H/N$; the inverse temperature of the m -th iterate follows from its energy moments, $\beta_m \simeq 2m/(N(L - \langle \hat{h} \rangle_m))$, and $C_v = \beta^2 \text{Var}(H)$. The fp32 GPU lane (`mtpq_f32.h`) halves the vector footprint for the frontier.

Trajectory aggregation (`ed/thermal/tpq_thermo.h`): samples' $(\beta_k, E_k, \text{var}_k)$ trajectories are interpolated at $\beta = 1/T$ and Welford-averaged; *absolute* entropy and free energy come from thermodynamic integration anchored at the exact infinite- T point:

$$\ln Z(\beta) = \ln D - \int_0^\beta \langle E \rangle(\beta') d\beta', \quad S = \beta \langle E \rangle + \ln Z, \quad F = -\ln Z/\beta, \quad (9)$$

with the $[0, \beta_1]$ head segment anchored at $\langle E \rangle(0) = \text{Tr } H/D$ (recovered for free from every sample's $\beta \approx 0$ first trajectory point). The absolute F_s is what makes per-sector free energies usable as Boltzmann weights in the $U(1)$ /spatial recombination.

KPM-DOS: Chebyshev moments of the rescaled Hamiltonian with Jackson damping and a Hutchinson stochastic trace; the moment and random-vector counts are forwarded knobs (a silently-defaulted $M=2048, R=20$ was one of the converter-fork casualties fixed in Stage 11a).

7.2 Ground-state dynamical structure factors

The zero-temperature spectral function of a probe O is

$$S_O(\omega) = \sum_n |\langle n | O | 0 \rangle|^2 \delta(\omega - (E_n - E_0)) = -\frac{1}{\pi} \text{Im} \langle \phi | (\omega + i\eta + E_0 - H)^{-1} | \phi \rangle, \quad |\phi\rangle = O|0\rangle, \quad (10)$$

evaluated as a continued fraction from the Lanczos coefficients of H seeded with $|\phi\rangle$:

$$\langle \phi | (z - H)^{-1} | \phi \rangle = \frac{\|\phi\|^2}{z - \alpha_0 - \frac{\beta_1^2}{z - \alpha_1 - \frac{\beta_2^2}{\ddots}}}. \quad (11)$$

In the symmetry-resolved pipeline, $|0\rangle$ is located by the star walk and solved *plain* in its momentum sector (residual-guarded); then $O|0\rangle$ is *scattered* into every raw destination sector of every reachable diagonal subspace ($n_{\uparrow} \pm \Delta$, parity halves, full space) through the rectangular representative apply (`CrossSectorOrbitObservable`): source orbits regenerate per group element, the destination projects through `index_and_projection`. Selection rules are *physical*: $\|\phi_{\text{dst}}\| < \varepsilon$ means the channel is forbidden; no selection-rule plumbing exists, and the weights obey $\sum_{\text{dst}} \|\phi_{\text{dst}}\|^2 = \|O|0\rangle\|^2$. For real H , a $-Q$ panel whose probe is the coefficient-conjugate of a computed $+Q$ probe is copied exactly: $\langle n|\bar{O}|0\rangle = \overline{\langle n|O|0\rangle}$ in a real eigenbasis.

Residual guards. Both GS re-solve sites in the cross-irrep bindings enforce $\|H\psi_0 - E_0\psi_0\|/\|\psi_0\| < 10^{-8}$ before any weight is computed — the orchestrator’s warn-not-fail contract otherwise returns a best-effort pair whose garbage the scattered weights and the continued fraction inherit silently:

```
template <class SectorView>
inline void require_gs_residual(SectorView& sec,
                               const std::vector<Complex>& psi0, double E0) {
    const std::size_t d = psi0.size();
    std::vector<Complex> hv(d, Complex(0.0, 0.0));
    sec.apply(psi0.data(), hv.data(), d);
    double num = 0.0, den = 0.0;
    for (std::size_t i = 0; i < d; ++i) {
        const Complex r = hv[i] - E0 * psi0[i];
        num += std::norm(r); den += std::norm(psi0[i]);
    }
    const double resid = (den > 0.0) ? std::sqrt(num / den)
                                   : std::numeric_limits<double>::infinity();
    if (!(resid < 1e-8)) throw std::runtime_error(/* loud, actionable */);
}
```

8 Backends, dispatch, and the GPU dimension floor

`select_backend(geometry, constraints)` (`ed/core/select_backend.h`) picks `{Cpu, Cuda, Mpi}` (with `MpiCuda` opt-in, never auto-selected): MPI when the process runs under `mpirun` and the operator is host-resident; otherwise CUDA when available, memory-fitting, allowed — and, since July 2026, only above a dimension floor:

```
struct BackendConstraints {
    bool allow_gpu = true; bool allow_mpi = true; bool allow_mpi_gpu = true;
    std::optional<std::size_t> gpu_mem_bytes;
    double fudge_factor = 8.0;
    /// Minimum problem dimension for the GPU AUTO-promotion. Below this,
    /// launch + transfer overhead makes the GPU strictly slower -- and on
    /// shared-GPU hosts tiny solves dispatched to a contended device have
    /// produced silently-wrong spectra. Explicit device='gpu' sets 0.
    std::size_t gpu_dim_floor = (std::size_t{1} << 14);
};
...
const bool dim_ok = op_is_device || geom.local_dim >= c.gpu_dim_floor;
if (device_mv && have_gpu && gpu_fits && c.allow_gpu && dim_ok) {
    return BackendVariant{std::make_unique<ed::matvec::CudaBackend>()};
}
```

Rationale: the recurring silently-wrong tiny-DSSF class (spectra $50\text{--}250\times$ too small, three sightings,

only ever when the pytest lane launched immediately after the C++ test sweep) traced to the ~ 40 -dimensional internal solves of the cross-irrep DSSF auto-dispatching to a WSL2 GPU exactly when the host had it contended. Tiny solves are also strictly slower on the device, so the auto-promotion was lose-lose. Explicit GPU requests bypass the floor at every surface (the one solve converter, the thermal converter and overrides, `qed.spectral(device='gpu')`, raw bindings setting the field); tiny-fixture GPU tests keep coverage by zeroing it. MPI is one story: `mpirun ED` — across-sector distribution (SectorDistributor: Burnside-weighted greedy packing of whole sectors onto ranks, rank-local solves) $\times$ the in-process `MpiBackend` for reductions.

9 One operator template, one parameter converter

9.1 The operator hierarchy

Historically four operator classes differed only in their basis. Today the hierarchy is

$$\text{LinearOperator} \leftarrow \text{Operator} \leftarrow \text{SubspaceOperator} \langle \text{BasisPolicy} \rangle,$$

with `FixedSzOperator` and `SectorOperator` as using-aliases (`FixedSzBasisPolicy` / `SymmetryBasisPolicy`). The basis axis is a POD *policy* consumed by the matvec kernels plus an owning *producer* (`FixedSzSubspace` / `SectorBasis`) selected by `SubspaceProducerTraits` $\langle$ `Policy` $\rangle$. After Stage 11c-2b the symmetry lane's backend factory is unconditional:

```
template <>
inline std::unique_ptr<ed::matvec::MatVecBackendBase>
SubspaceOperator<ed::matvec::basis::SymmetryBasisPolicy,
    ed::matvec::MemorySpace::Host>::make_backend() const {
    // ONE representation: the CSR-free RepSectorData drives both
    // sub-modes (RepStream walk; budget-gated RepReducedCsr).
    const ed::symmetry::RepSectorData& rd = producer_.ensureRepData();
    if (!rd.usable())
        throw std::runtime_error(/* construction-bug invariant */);
    return ed::matvec::make_cpu_rep_symmetry_backend<...>(rd);
}
```

The factory surface is `ed::make_operator(OperatorSpec)`; symmetry specs return one `SectorOperator` per surviving sector through the tagged factory (`make_sector_operators_tagged`), iterated as a `SectorSetView` by the CLI sector loop, the Python bindings, and the SectorDistributor (which greedily packs whole sectors onto MPI ranks by their Burnside dimensions and solves rank-locally).

9.2 The one converter

Parameters cross the Python/C++ boundary through a single function, `ed_adapter::toSolveOptions` (`ed/core/ed_config_adapter.h`), bound as `_core.ed_params_to_solve_options`; the two callers' deliberate semantic differences are *flags*, not divergent copies:

```
inline ed::workflows::SolveOptions
toSolveOptions(const EParameters& params,
    ::DiagonalizationMethod method,
    bool auto_method = false,
    bool wire_backend = false, // Python: device= PINS the backend
    bool allow_infeasible = false) {
    ...
    if (auto_method) opts.method = SM::Auto;
```

```

    if (wire_backend) {
        opts.backend.allow_gpu = params.use_gpu;
        opts.backend.allow_mpi = params.use_mpi;
        // Python vetted the GPU choice (explicit device='gpu' or its own
        // dim >= 2^14 heuristic): the C++ auto-promotion floor must not
        // second-guess it.
        if (params.use_gpu) opts.backend.gpu_dim_floor = 0;
    }
    opts.allow_infeasible = allow_infeasible;
    ...
}

```

The thermal converter (python/qed/_params.py, no C++ twin) follows the same rules and forwards every method knob strictly — the historical fork between two Python copies (silent default-to-FTLM; dropped KPM moment counts, a $\sim 250\times$ cost amplifier) is the incident that motivated the whole parameter consolidation.

10 Cross-cutting guards and content-addressed caching

10.1 Correctness never rides on bookkeeping

- **Covering sum rule** (2): after every fold and projection, pooled multiplicities must tile the subspace dimension.
- **Monomial probe**: every accepted M_p passes the numerical $[M_p, H_{k_0}] = 0$ random-vector test; the isotypic covering $\sum_{\sigma} |W_{\sigma}| d_{\sigma} = \dim k_0$ is checked before any block replaces the plain solve.
- **Residual gates**: the engine’s ground-state factory throws on a stale Ritz pair; both cross-irrep GS sites run `require_gs_residual`; the lowest- k window is stationarity-gated (§6).
- **Oracles**: routed verbs $\equiv$ the retired monolithic SAB engine $\equiv$ dense references, pinned across `test_little_group*.py`, `test_point_group_routing.py`, and multiset window pins. One local gate (`scripts/check.local.sh`) runs every CI lane.

10.2 Hash quality is a correctness concern

Content-keyed registries (orbit tables, GPU mirrors, SAB cache) briefly keyed on a word-XOR FNV digest. That mix is *degenerate under character sign patterns*: on a $\mathbb{Z}_8$ ring, $\chi_{k+4}(g) = (-1)^g \chi_k(g)$, and XOR-folding the words of two such character vectors collides — observed as GPU sectors 5/7 silently reusing the mirrors of 1/3. Two independent fixes, both required: (i) every key material word passes through the `splitmix64` avalanche,

```

z = (z ^ (z >> 30)) * 0xbf58476d1ce4e5b9ULL;
z = (z ^ (z >> 27)) * 0x94d049bb133111ebULL;
return z ^ (z >> 31);

```

and (ii) *no hit is trusted*: mirror slots verify a full identity fingerprint; orbit-table registry and disk hits are physically re-verified (`orbit_table_consistent`: sampled representatives checked for subspace membership and canonical minimality under the caller’s group), with loud eviction and rebuild on failure.

11 The Stage-11 consolidation ledger

Measured against the ideal pipeline, the tree carried four Lanczos generations, five parameter layers, two basis-construction lanes, two-plus matvec representations per backend, and a 6.6 kLOC distributed-operator family with a single consumer. Nine CI-green commits removed the duplication:

Stage	The one implementation	Deleted
11a + tail	<code>_params.py</code> $\Rightarrow$ <code>toSolveOptions</code>	forked thermal converter; Python solve-converter twin
11b	<code>lanczos_kernel<Backend></code>	<code>TPQ.cpp</code> , <code>dynamics.cpp</code> , <code>time_evolution.h</code> , ...
11c-1/2a	lazy rep-first builders	eager orbit-CSR builders; <code>adopt</code> bridge; budget fork
11c-2b	CSR-free rep kernel (host+device)	CPU orbit backend; GPU orbit mirror; device orbit lane
11d-prep	<code>SubspaceOperator<Policy></code>	(verified real); stale prose; straggler gates
11d	<code>mpirun ED = SectorDistributor $\times$ MpiBackend</code>	<code>ed::distributed</code> family; launcher; <code>qed.mpi</code>

Env-gate hygiene follows the same ledger (`ed/symmetry/env_gates.h` / `qed.debug_env()`): live gates are one X-macro list; retired gates (`ED_SYM_REP`, `ED_GPU_SYMMETRY_REP`, `ED_SYM_LAZY_SECTORS`, ...) are named in its retirement comment rather than lingering as dead reads.

12 Validation numbers

- **Little-group scaling** (ring ground state, full group): $N=12$: 0.005 s vs 0.045 s monolithic; $N=16$: 0.03 vs 7.1 s; $N=20$: 0.5 vs 317 s ($626\times$), $\Delta E_0 = 0$.
- **Phase profile** ($N=22$ D_{22} ring, warm caches, `ED_SYM_PROFILE=1`): sector data 0.006 s; monomials + isotypic 0.33 s; block solves 2.9 s (dominant). 44 extended irreps fold to 24 stars.
- **GPU rep mirror** ($N=24$, end-to-end): 4.8 s vs the retired orbit-CSR mirror's 18.2 s.
- **Ghost-Ritz case** (4×4 J_1-J_2 , $J_2=0.15$, $n_{\uparrow}=8$, $k=10$): pre-fix multiset error 1.4 (every block at its 60-iteration cap); post-fix 1.3×10^{-13} ; suites 34/34.
- **Memory**: shared rank tables collapse the per-sector reverse lookup from ~ 77 GB aggregate to ~ 4.8 GB at $N=32$ half filling; reduced-CSR budget gate keeps $N=36$ matrix-free with no configuration.
- **After the dimension floor**: the first fully-green local gate on the development host (381/381 ctest, all pytest, tour) — the previously load-flaky spectral tests no longer touch the contended GPU.

Provenance

Companion documents: `UNIFIED_STACK.md` (layer map), `SYMMETRY.V2_DESIGN.md` §4 (stage-by-stage migration record, rows 9a–11d), `SYMMETRY.md` (mechanism reference), `SCALING.md` (memory envelope). Standard references for the numerical background: C. C. Paige's analysis of Lanczos in finite precision and Cullum–Willoughby on ghost identification; Jaklič–Prelovšek for FTLM; Sugiura–Shimizu for thermal pure quantum states; Weiße et al. for the kernel polynomial method.